

Experiment # 03

Solving a regression problem

Objectives:

- Overview of regression
- Simple linear regression
- Multiple linear regression
- Example
- Lab Tasks

1. Overview of regression

Regression is a process of finding the correlations between dependent and independent variables. It helps in predicting the continuous variables such as prediction of Market Trends, prediction of House prices, etc.

The task of the Regression algorithm is to find the mapping function to map the input variable(x) to the continuous output variable(y).

Example: Suppose we want to do weather forecasting, so for this, we will use the Regression algorithm. In weather prediction, the model is trained on the past data, and once the training is completed, it can easily predict the weather for future days.

- Simple Linear Regression
- Multiple Linear Regression
- Polynomial Regression
- Support Vector Regression
- Decision Tree Regression
- Random Forest Regression

2. Simple linear regression

Simple linear regression is an approach for predicting a response using a single feature. It is assumed that the two variables are linearly related. Hence, we try to find a linear function that predicts the response value(y) as accurately as possible as a function of the feature or independent variable(x).

Let us consider a dataset where we have a value of response y for every feature x :

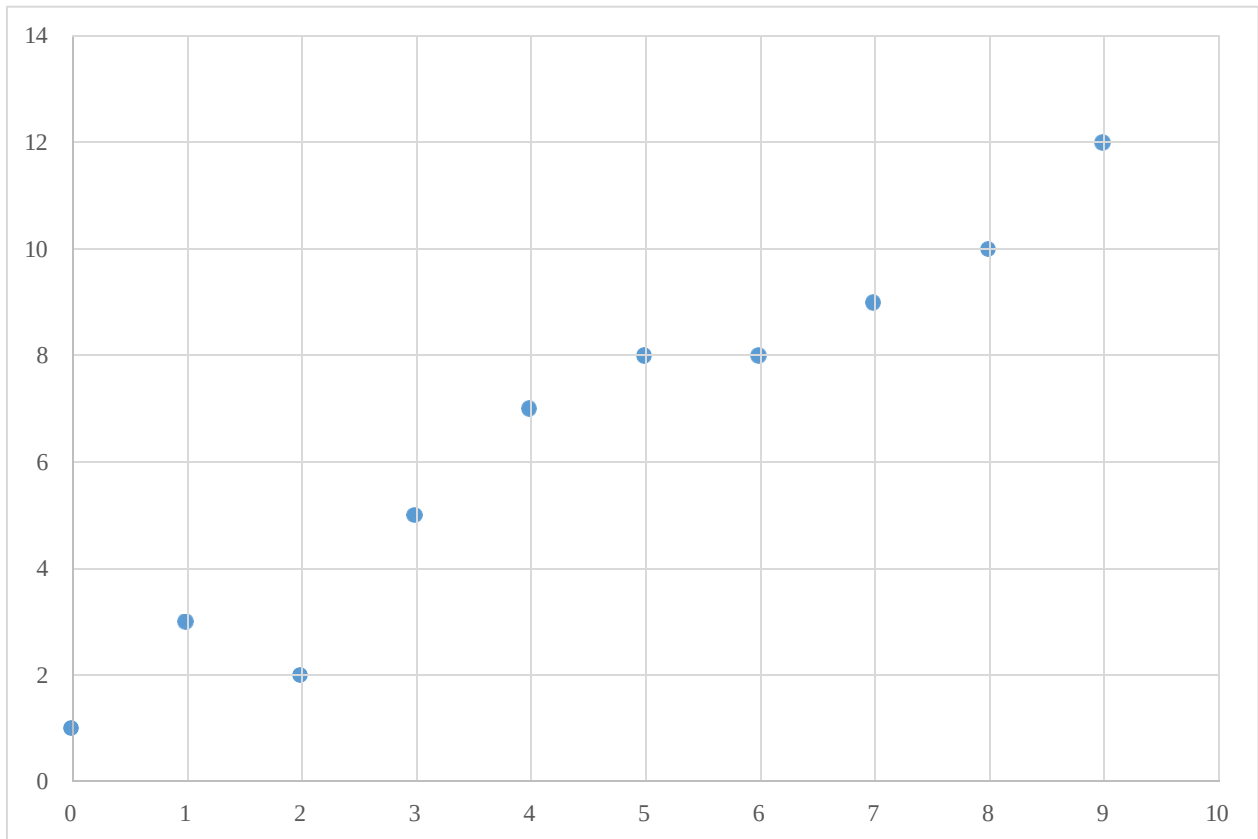
Department of Computer Engineering
CS-302 Artificial Intelligence

x	0	1	2	3	4	5	6	7	8	9
y	1	3	2	5	7	8	8	9	10	12

For generality, we define:

- x as feature vector, i.e $x = [x_1, x_2, \dots, x_n]$,
- y as response vector, i.e $y = [y_1, y_2, \dots, y_n]$

For n observations (in above example, $n=10$) a scatter plot of the above dataset looks like:



Now, the task is to find a line that fits best in the above scatter plot so that we can predict the response for any new feature values. (i.e. a value of x not present in a dataset). This line is called a regression line.

The equation of regression line is represented as:

$$h(x_i) = \beta_0 + \beta_1 x_i$$

Here,

- $h(x_i)$ represents the predicted response value for i th observation.
- β_0, β_1 are regression coefficients and represent y-intercept and slope of regression line respectively.

To create our model, we must “learn” or estimate the values of regression coefficients β_0 and β_1 . And once we’ve estimated these coefficients, we can use the model to predict responses.

In this lab experiment, we are going to use the principle of *Least Squares*.
Now consider:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i = h(x_i) + \varepsilon_i \Rightarrow \varepsilon_i = y_i - h(x_i)$$

Here, ε_i is a residual error in i th observation. So, our aim is to minimize the total residual error. We define the squared error or cost function, J as:

$$J(\beta_0, \beta_1) = \frac{1}{2n} \sum_{i=1}^n \varepsilon_i^2$$

Our task is to find the value of β_0 and β_1 for which (β_0, β_1) is minimum.

Without going into the mathematical details, we present the result here:

$$\beta_1 = \frac{SS_{xy}}{SS_{xx}}$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

where SS_{xy} is the sum of cross-deviations of y and x :

$$SS_{xy} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) = \sum_{i=1}^n y_i x_i - n \bar{x} \bar{y}$$

and SS_{xx} is the sum of squared deviations of x :

$$SS_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2 = \sum_{i=1}^n x_i^2 - n \bar{x}^2$$

Note: The complete derivation for finding least squares estimates in simple linear regression can be found [here](#).

2.1. Code: Python implementation of above technique on our small dataset

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, r2_score

# Function to estimate regression coefficients
def estimate_coef(x, y):
    n = np.size(x) # Number of data points
    m_x, m_y = np.mean(x), np.mean(y) # Mean of x and y

    # Cross-deviation and deviation about x
    SS_xy = np.sum(y * x) - n * m_y * m_x
    SS_xx = np.sum(x * x) - n * m_x * m_x

    # Regression coefficients
    b_1 = SS_xy / SS_xx
    b_0 = m_y - b_1 * m_x

    return (b_0, b_1)

# Function to predict new data points
def predict_new_values(x_new, b):
    return b[0] + b[1] * x_new # Regression formula Y = b0 + b1*X

# Function to evaluate regression performance
def evaluate_regression(y_actual, y_predicted):
    mse = mean_squared_error(y_actual, y_predicted)
    r2 = r2_score(y_actual, y_predicted)
    return mse, r2

# Function to plot regression line, original data, and new predictions
def plot_regression_line(x, y, b, x_new=None, y_new=None):
    # Scatter plot for actual data
    plt.scatter(x, y, color="m", marker="o", s=30, label="Actual data")

    # Extend regression line to cover new X values
    x_extended = np.append(x, x_new) if x_new is not None else x
    y_pred_extended = b[0] + b[1] * x_extended
    plt.plot(x_extended, y_pred_extended, color="g", linestyle="--",
             label="Regression line")

    # Plot predicted new data points if provided
```

Department of Computer Engineering
CS-302 Artificial Intelligence

```
if x_new is not None and y_new is not None:
    plt.scatter(x_new, y_new, color="r", marker="x", s=100,
label="Predicted points")

# Labels, legend, and title
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Linear Regression')
plt.legend()

# Show plot
plt.show()

# Main function
def main():
    # Training data
    x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
    y = np.array([1, 3, 2, 5, 7, 8, 8, 9, 10, 12])

    # Estimate coefficients
    b = estimate_coef(x, y)
    print(f"Regression Coefficients: b_0 = {b[0]:.2f}, b_1 = {b[1]:.2f}")

    # Predicting values for new data points
    x_new = np.array([10, 11, 12]) # New X values
    y_new = predict_new_values(x_new, b)

    # Printing predicted values
    print("Predicted values for new X points:")
    for i in range(len(x_new)):
        print(f"X = {x_new[i]}, Predicted Y = {y_new[i]:.2f}")

    # Compute predicted values for original dataset (for evaluation)
    y_pred_original = predict_new_values(x, b)

    # Evaluate regression performance
    mse, r2 = evaluate_regression(y, y_pred_original)
    print(f"Mean Squared Error (MSE): {mse:.2f}")
    print(f"R-squared Value (R²): {r2:.2f}")

    # Plot regression line with actual data and predicted new points
    plot_regression_line(x, y, b, x_new, y_new)
```

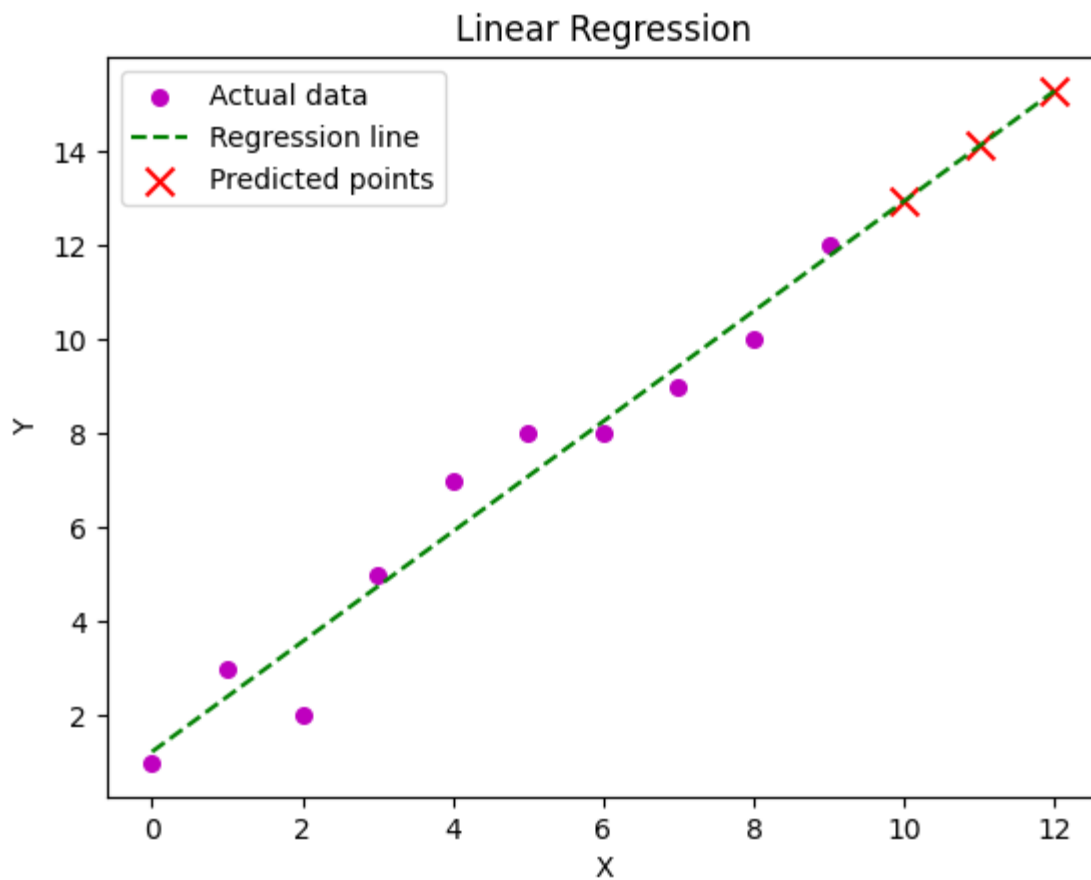
Department of Computer Engineering
CS-302 Artificial Intelligence

```
# Running the script
if __name__ == "__main__":
    main()
```

Output

```
Regression Coefficients: b_0 = 1.24, b_1 = 1.17
Predicted values for new X points:
X = 10, Predicted Y = 12.93
X = 11, Predicted Y = 14.10
X = 12, Predicted Y = 15.27
Mean Squared Error (MSE): 0.56
R-squared Value (R2): 0.95
```

Output Plot



3. Task

Write the above-mentioned program for different datasets and perform different types of regression techniques and plot regression line with actual and new datapoints with predicted output.